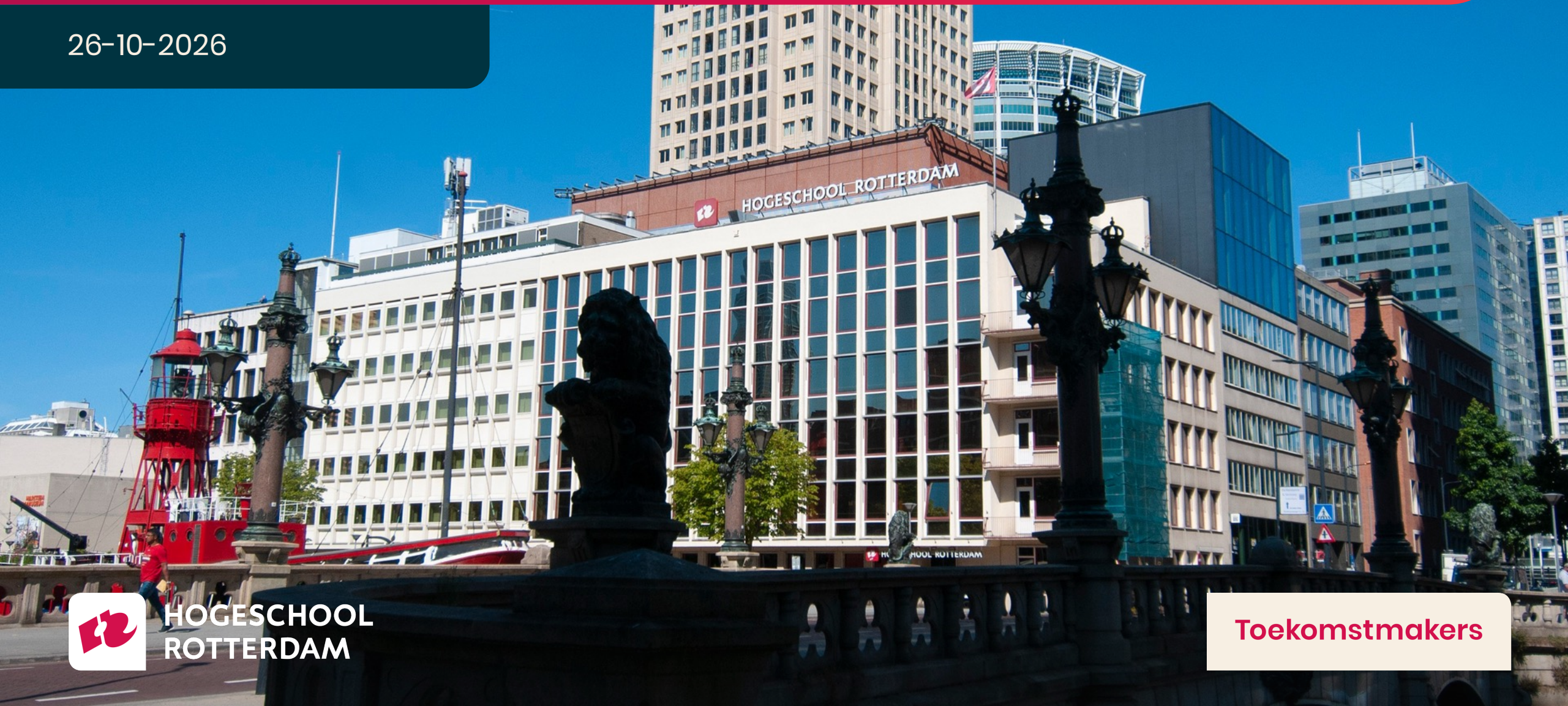
Technische Informatica: High-performance Computing

Les 19: Textuur & belichting

26-10-2026

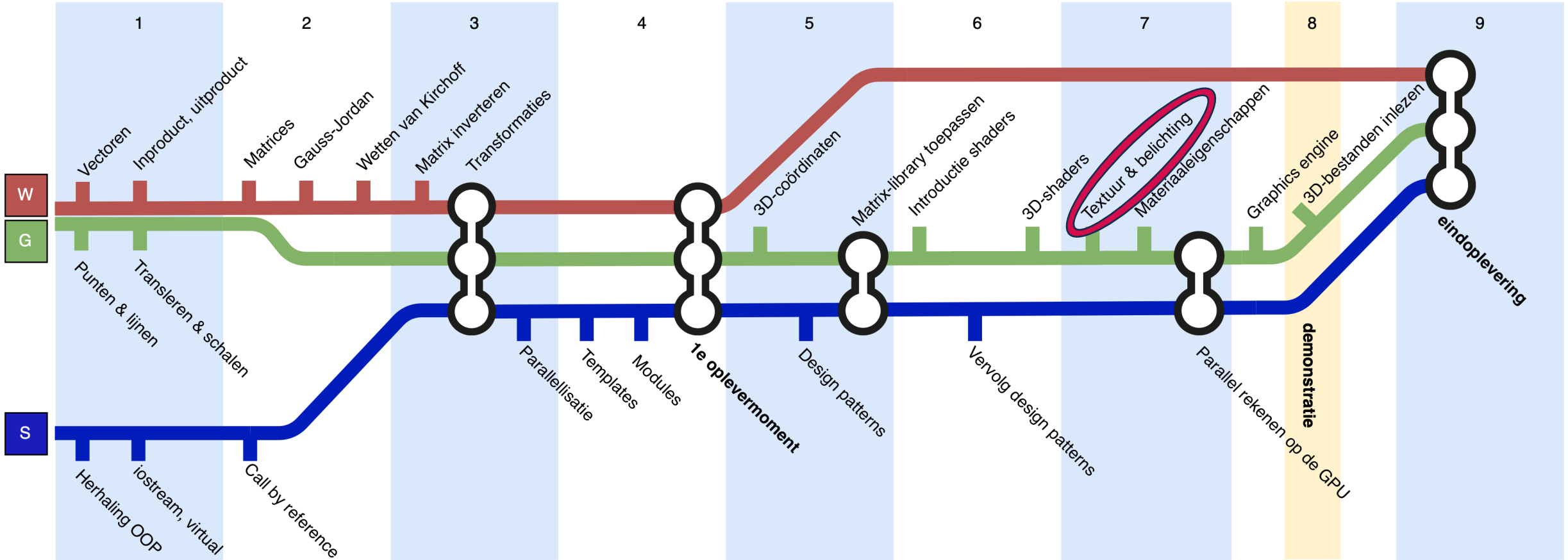


HOGESCHOOL ROTTERDAM

Toekomstmakers

Quiz

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
Punten & lijnen
Transleren & schalen
Herhaling OOP
iostream, virtual
Call by reference
Parallellisatie
Templates
Modules
1e oplevermoment
3D-coördinaten
Design patterns
Matrix-library toepassen
Introductie shaders
Vervolg design patterns
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Parallel rekenen op de GPU
Graphics engine
3D-bestanden inlezen
demonstratie
eindoplevering

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.

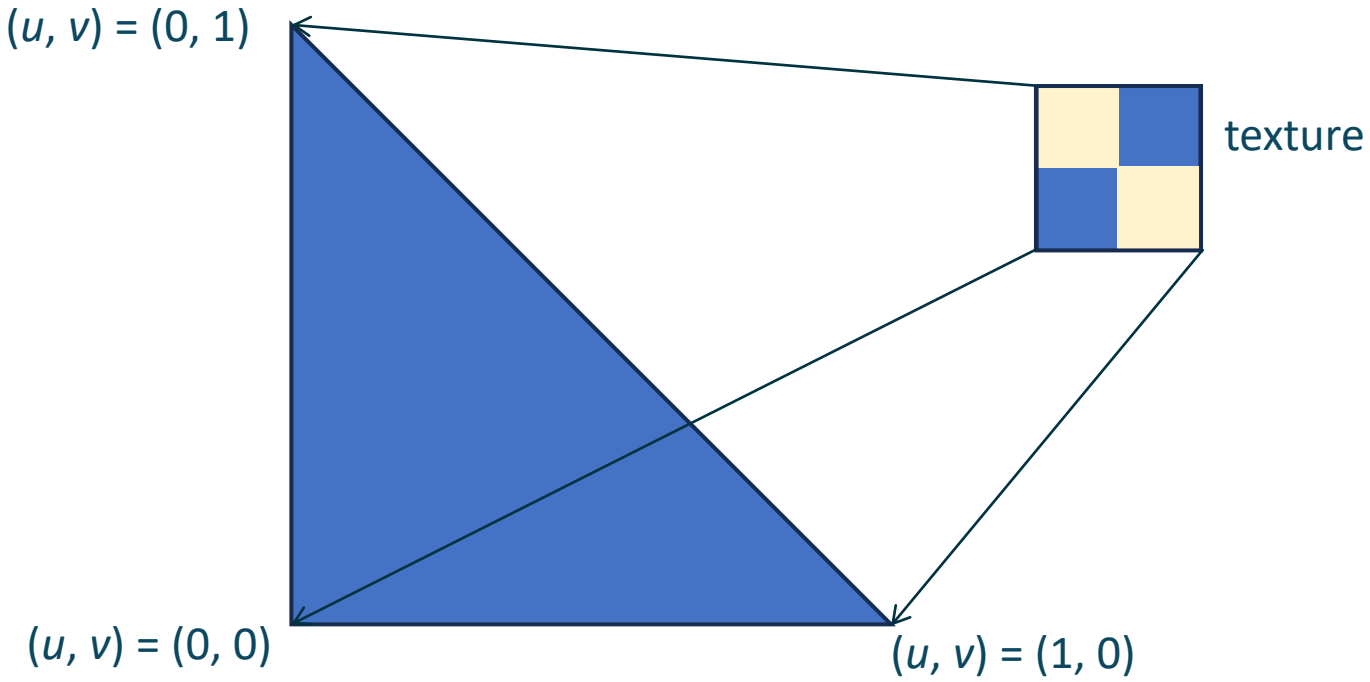
Planning

1. Bespreken opdracht vorige les
2. Textuur
3. Belichting



Texture toevoegen

- Meestal willen we meer details afbeelden op het oppervlak ⇒ *texture mapping*.
- De texture staat in een array van RGB-waarden (bijv. een foto).
- **Probleem**: waar op het oppervlak moet de texture komen?
- **Oplossing**: Voeg texture-coördinaten *u*, *v* toe aan iedere vertex.



Texture toevoegen

- Met texture-coördinaten groter dan 1 wordt de texture als een tegelpatroon herhaald.
- Vertices:

```
GLfloat vertices[] = {
     1,  1,  1, 5, 0, // front top right
     1, -1,  1, 5, 5, // front bottom right
    -1, -1,  1, 0, 5, // front bottom left
    -1,  1,  1, 0, 0, // front top left
     1,  1, -1, 0, 0, // back top right
     1, -1, -1, 0, 5, // back bottom right
    -1, -1, -1, 5, 5, // back bottom left
    -1,  1, -1, 5, 0, // back top left
};
```



Texture toevoegen

- **Vertex shader** krijgt texture-coördinaten binnen als *attribute* en geeft deze door aan de fragment shader met met een `out` variable:

```
#version 330 core
in vec3 pos;
in vec3 col;
in vec2 tex;
out vec3 color = col;
out vec2 texCoord = tex;
      ⋮
```



Texture toevoegen

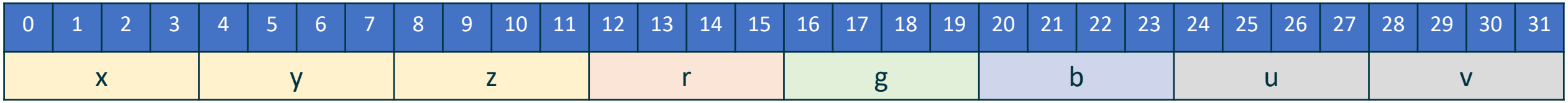
- **Fragment shader** krijgt texture-coördinaten binnen als *attribute* met dezelfde naam als de `out` variable in de vertex shader.
- De texture zelf is een *uniform* van type `sampler2D`.
- Met de functie `texture()` wordt de juiste plek opgezocht:

```
#version 330 core
in vec3 color;
in vec2 texCoord;
out vec4 fragColor;
uniform sampler2D textureSampler;
void main() {
    fragColor = texture(textureSampler, texCoord) * vec4(color, 1.0);
}
```



Texture toevoegen

- **Attributes** `pos`, `col` en `tex` worden gevuld vanuit de vertex buffer.
- 1 vertex:




```
GLint texLocation = glGetAttribLocation(program, "tex");
glVertexAttribPointer(texLocation, 2, GL_FLOAT, GL_FALSE, 32, (void *)24);
glEnableVertexAttribArray(texLocation);
```

- 2 getallen voor texture-coördinaten
- Totale *stride*: 32 bytes (8 32-bit floats)
- *Offset* voor texture-coördinaten: 24 bytes (6 32-bit floats)



Texture toevoegen

- Maak een *texture buffer* om de texture in op te slaan:

```
GLuint texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);
```

- Koppel de texture-array aan de texture buffer:

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,
GL_UNSIGNED_BYTE, pixels);
```

  - Breedte & hoogte
  - Texture-array



- Geef aan hoe de texture geschaald moet worden:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
```



Opdracht 1: Texture

- **Zie instructies op GitLab**.

- Bestudeer het gegeven voorbeeldprogramma.
- Pas de texture-coördinaten van de vertices aan zodat het plaatje maar 1 × op de zijden van de kubus geplakt wordt.
- Vervang de gegenereerde texture door een plaatje uit een bestand.

- Stel, je hebt meerdere kubussen met dezelfde texture. Laad je dan voor iedere kubus de texture opnieuw in het geheugen? Dit neemt een hoop geheugenruimte in beslag.



Pauze (15min)

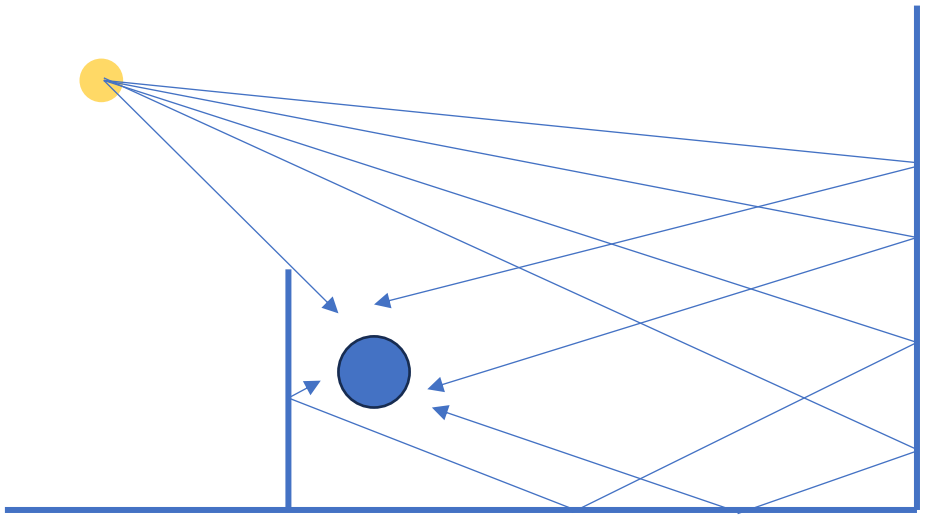
Belichting

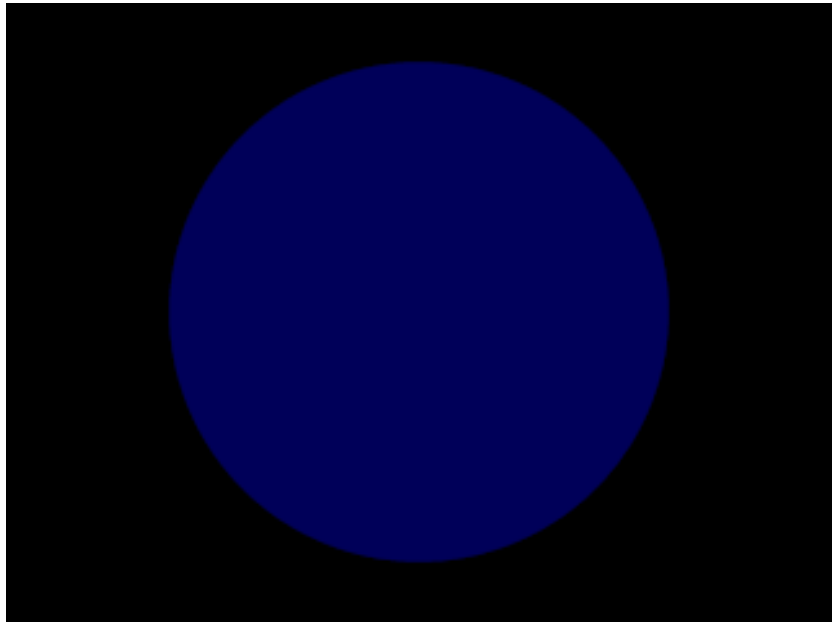
- We zien alleen objecten die licht van een lichtbron reflecteren in ons oog.
- Daarom zijn lichtbronnen essentieel voor een realistische 3D-voorstelling.
- Verschillende soorten belichting:
  - Ambient lighting
  - Diffuse lighting
    - Directional
    - Positional
  - Specular lighting



Ambient lighting

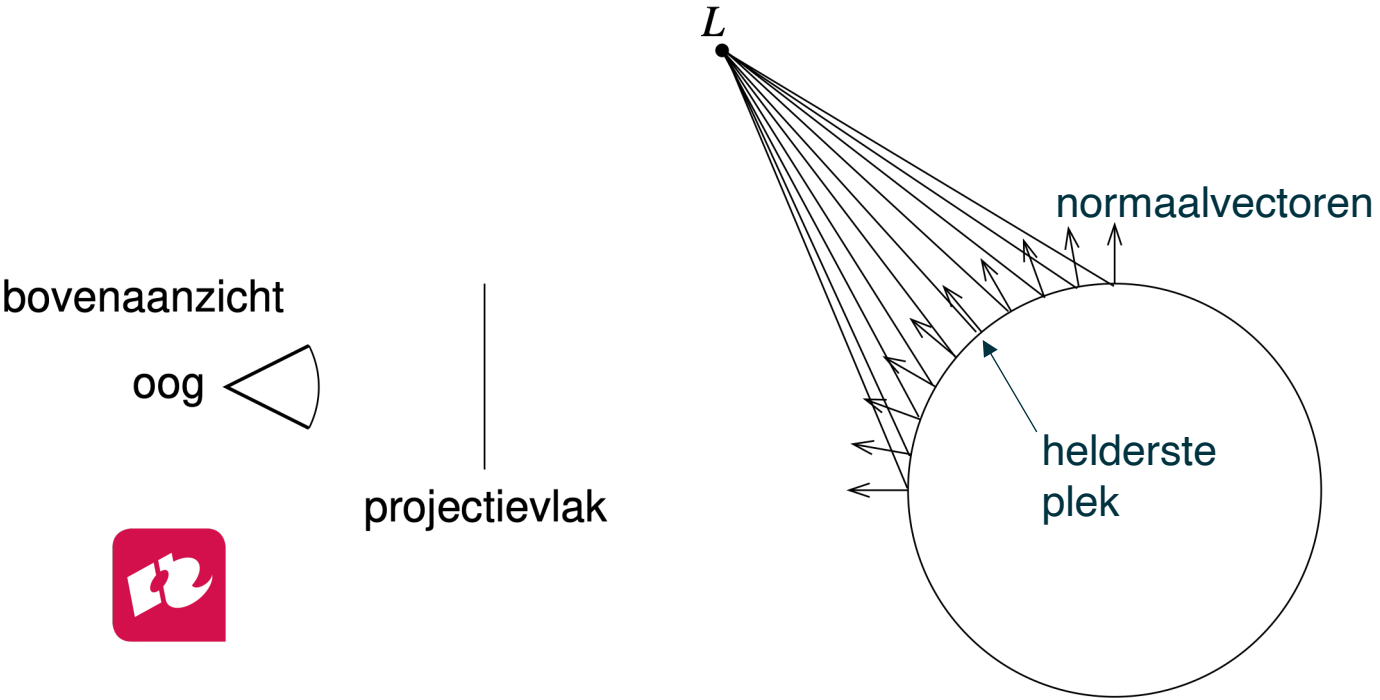
- Licht dat overal “aanwezig” (ambient) is en van alle kanten komt.
- In realiteit: resultaat van heel veel reflecties van allerlei oppervlakken.
- Te complex om precies te modelleren ⇒ simpele benadering.

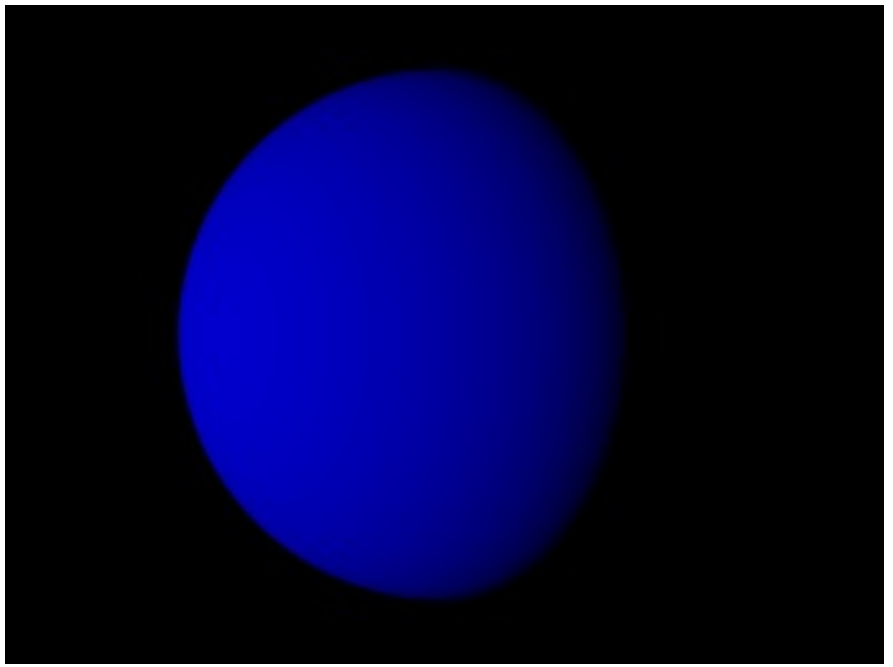




Diffuse lighting

- **Lambertiaans belichtingsmodel**
- Hoe meer een oppervlak naar de lichtbron is toegekeerd, hoe sterker het verlicht wordt.
- Hoe beter de *normaalvector* overeenkomt met de *omgekeerde lichtrichtingsvector*, hoe sterker verlicht.

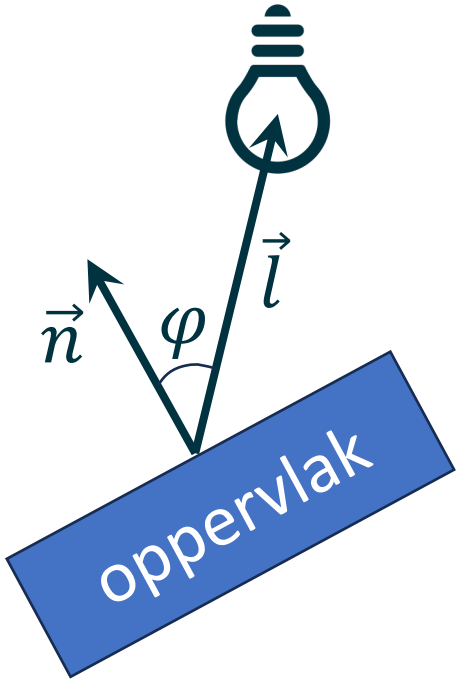




Diffuse lighting

- **Lichtintensiteit I is het hoogste** als de normaalvector van het oppervlak $\vec{n}$ en de omgekeerde lichtvector $\vec{l}$ in dezelfde richting staan.
- We kunnen hiervoor het **inproduct** gebruiken van de twee vectoren:

$$I \propto \vec{n} \cdot \vec{l} = |\vec{n}| \, \left|\vec{l}\right| \cos \varphi$$



- In GLSL is hiervoor de functie `dot()`.
- Om de berekening te laten kloppen moeten we met **eenheidsvectoren** werken (lengte 1).
- In GLSL is hiervoor de functie `normalize()`.

Directional lighting

- Voor *directional lighting* gebruiken we een **vaste lichtvector** (de lichtstralen komen evenwijdig binnen).
- Fragment shader:

```
#version 330 core
in vec3 normal;
vec3 reverseLightDirection = normalize(vec3(2.0f, 1.2f, 1.0f));
vec4 baseColor = vec4(1.0f, 1.0f, 1.0f, 1.0f);
out vec4 fragColor;
void main() {
    vec3 normalizedNormal = normalize(normal);
    float lightness = dot(normalizedNormal, reverseLightDirection);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```



Directional lighting

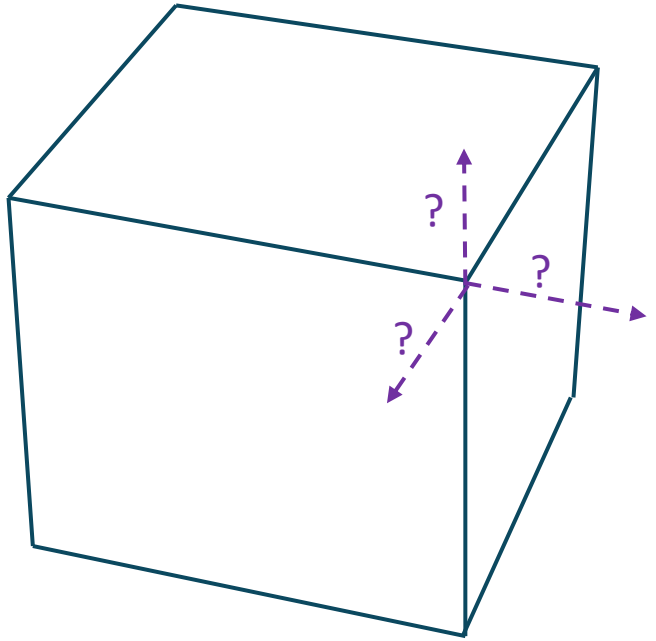
- We hebben dus de normaalvector nodig van ieder fragment.
- Deze wordt doorgegeven door de **vertex shader**.
- Zonder translatie en projectie, alleen rotatie:

```
#version 330 core
in vec3 pos;
in vec3 nor;
out vec3 normal;
⋮
void main() {
    ⋮
    gl_Position = projection * translation * rotation * rotationY * vec4(pos, 1.0);
    normal = mat3(rotation * rotationY) * nor;
}
```



Directional lighting

- We moeten dus bij iedere vertex een normaalvector toevoegen.
- **Probleem**: wat is de normaalvector van een hoekpunt?



- **Oplossing**: voor ieder vlak van de kubus aparte vertices.



Directional lighting

- Totaal 6 vlakken × 2 driehoeken × 3 hoekpunten = 36 vertices:

```
GLfloat vertices[] = {
  // front side
   1,  1,  1, 0, 0, 1, // front top right
   1, -1,  1, 0, 0, 1, // front bottom right
  -1, -1,  1, 0, 0, 1, // front bottom left
  -1, -1,  1, 0, 0, 1, // front bottom left
  -1,  1,  1, 0, 0, 1, // front top left
   1,  1,  1, 0, 0, 1, // front top right
  // right side
   1,  1,  1, 1, 0, 0, // front top right
   1, -1,  1, 1, 0, 0, // front bottom right
   1, -1, -1, 1, 0, 0, // back bottom right
```

Zelfde hoekpunt, andere normaalvector

enz. enz.



Directional lighting

- Voorzie nor-*attribute* van data:

```
GLint norLocation = glGetAttribLocation(program, "nor");
glVertexAttribPointer(norLocation, 3, GL_FLOAT, GL_FALSE, 24, (void *)12);
glEnableVertexAttribArray(norLocation);
```

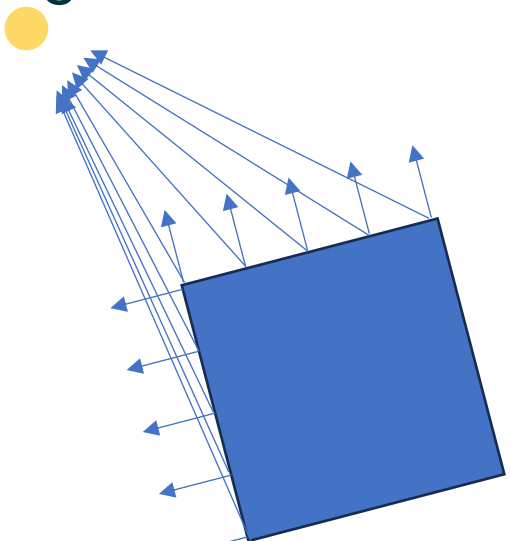
- Vertices worden niet hergebruikt, dus geen ElementArrayBuffer met edges meer nodig.
- Tekeninstructie wordt:

```
glDrawArrays(GL_TRIANGLES, 0, sizeof(vertices) / 6 / sizeof(GLfloat));
```



Positional lighting

- **Probleem**: Directional lighting niet heel realistisch.
  - Alle lichtstralen evenwijdig.
  - Kan alleen als lichtbron oneindig ver weg is.
- **Oplossing**: Positional lighting
  - Lichtbron met een specifieke positie in de 3D-ruimte.
  - `reverseLightDirection` wijst naar die positie.
  - Hangt af van de plek op het oppervlak.
  - Voor ieder fragment nodig: normaalvector, positie in 3D-ruimte.





Positional lighting

- Fragment shader:

```
#version 330 core
in vec3 normal;
in vec3 fragPos;
vec4 baseColor = vec4(1.0f, 1.0f, 1.0f, 1.0f);
vec3 lightPos = vec3(0.2f, 1.2f, 0.0f);
out vec4 fragColor;
void main() {
    vec3 normalizedNormal = normalize(normal);
    vec3 reverseLightDirection = normalize(lightPos - fragPos);
    float lightness = dot(normalizedNormal, reverseLightDirection);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```



Positional lighting

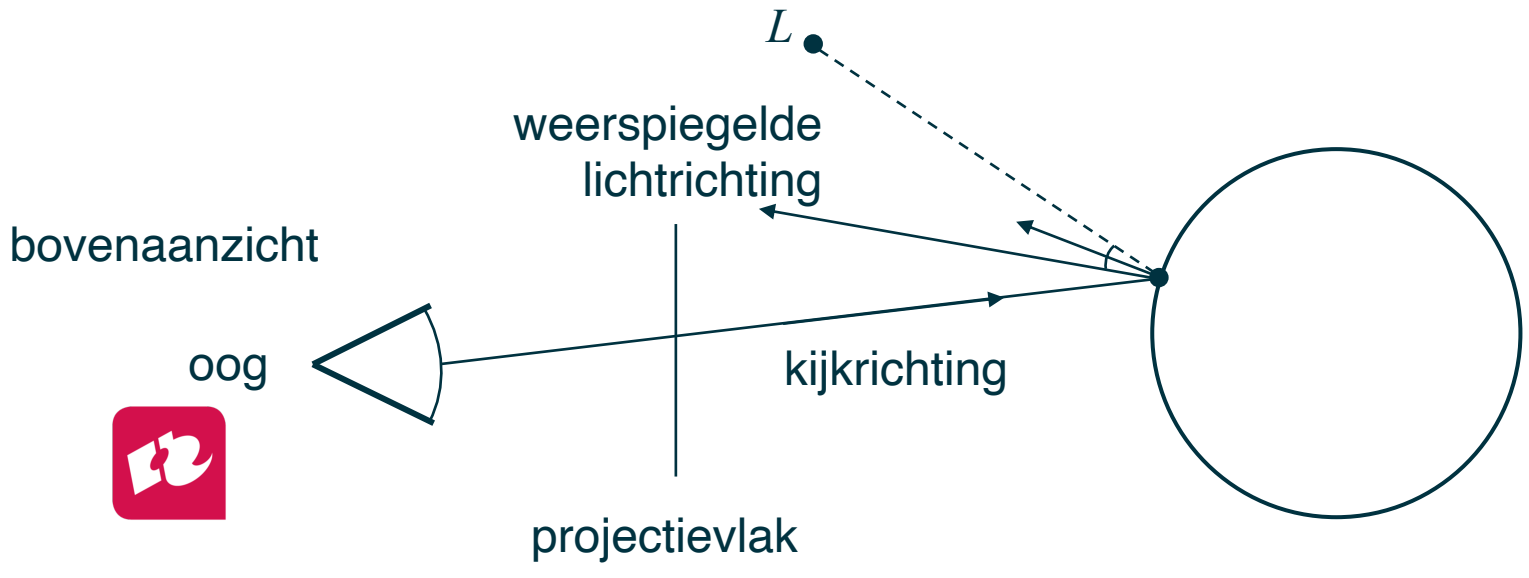
- Vertex shader geeft fragment-positie door aan fragment shader.
- Met rotatie en translatie, zonder projectie:

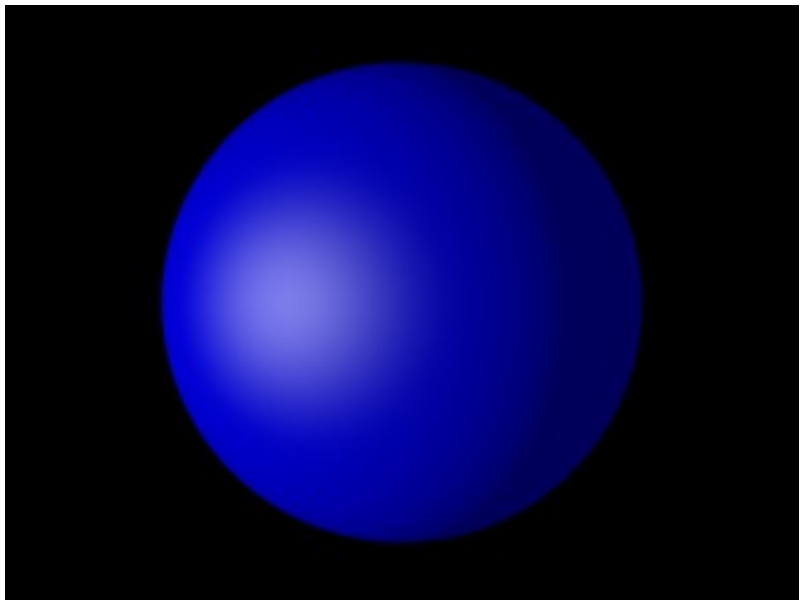
```
#version 330 core
in vec3 pos;
in vec3 nor;
out vec3 normal;
out vec3 fragPos;
⋮
void main() {
    ⋮
    gl_Position = projection * translation * rotation * rotationY * vec4(pos, 1.0);
    normal = mat3(rotation * rotationY) * nor;
    fragPos = vec3(translation * rotation * rotationY * vec4(pos, 1.0));
}
```



Specular lighting

- Glimmende oppervlakken weerspiegelen het licht (*specula* = spiegel).
- Hierdoor onstaat een lichtvlek op het oppervlak.
- De kleur van de vlek is de kleur van de lichtbron.
- Hoe beter de *kijkrichting* overeenkomt met de *omgekeerde weerspiegelde lichtrichting*, hoe sterker verlicht.

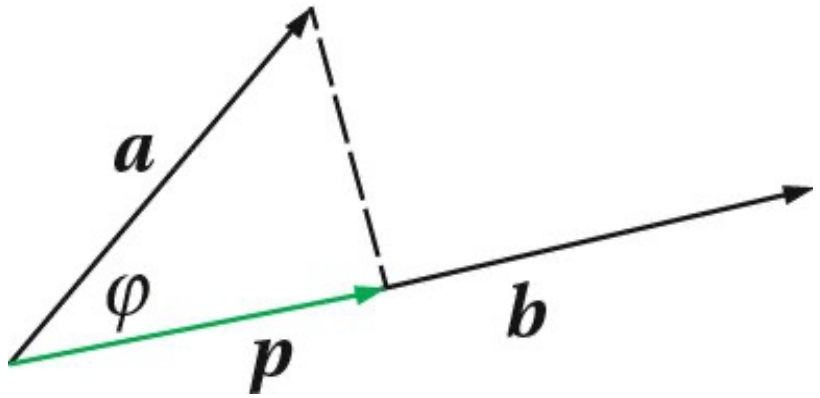




Nodig: projectie van een vector op een andere vector

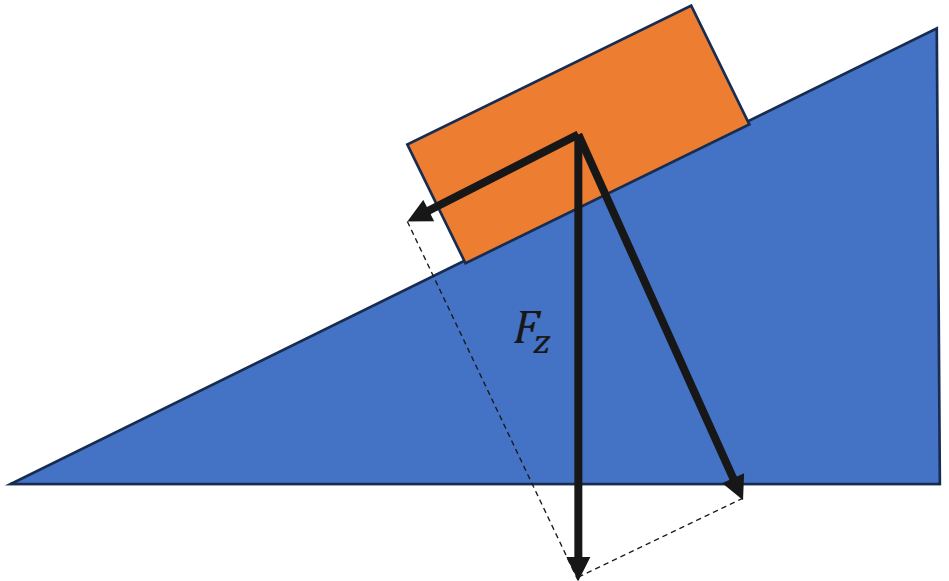
Projectie $\vec{p}$ van vector $\vec{a}$ op vector $\vec{b}$

- $\vec{p}$ heeft dezelfde richting als $\vec{b}$
- Lengte $\vec{p}$: $|\vec{p}| = |\vec{a}| \cos\varphi = |\vec{a}| \frac{\vec{a}\cdot\vec{b}}{|\vec{a}||\vec{b}|} = \frac{\vec{a}\cdot\vec{b}}{|\vec{b}|}$
- $\vec{p} = \frac{|\vec{p}|}{|\vec{b}|}\vec{b} = \frac{\vec{a}\cdot\vec{b}}{|\vec{b}|^2}\vec{b} = \frac{\vec{a}\cdot\vec{b}}{\vec{b}\cdot\vec{b}}\vec{b}$



Andere toepassingen:

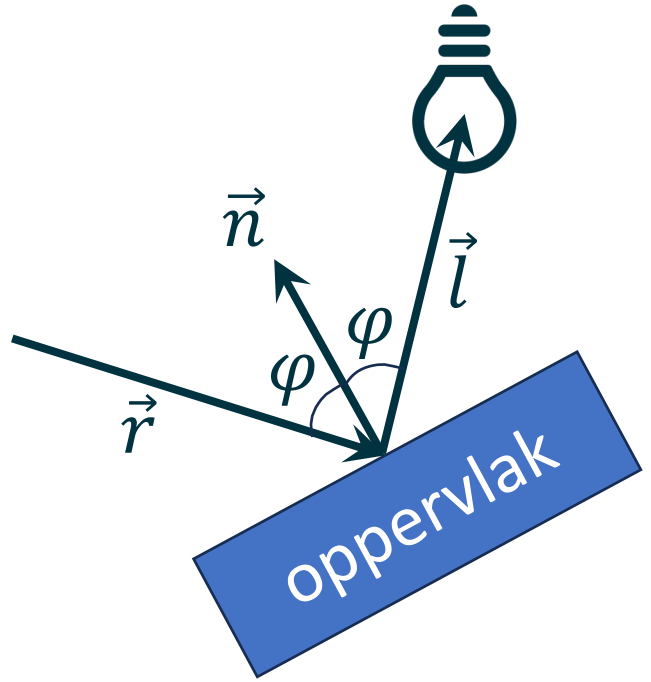
- ontbinden van krachten
- “Schaduw”



Specular lighting

- We hebben dus de *omgekeerde weerspiegelde lichtvector* $\vec{r}$ nodig.

$$\vec{r} = \vec{l} - 2(\vec{n} \cdot \vec{l})\vec{n}$$



- In GLSL is hiervoor de functie `reflect()`.
- We kunnen nu de omgekeerde weerspiegelde lichtvector vergelijken met de *kijkrichting*.
- Dus: inproduct van $\vec{r}$ met kijkrichting.
- Kijkrichting: de eenheidsvector in de richting van de oorsprong naar de positie op het oppervlak: `normalize(fragPos)`.

Specular lighting

- **Fragment shader**:

```
⋮
void main() {
    vec3 normalizedNormal = normalize(normal);
    vec3 reverseLightDirection = normalize(lightPos - fragPos);
    vec3 reflectedDirection = reflect(reverseLightDirection,
      normalizedNormal);
    vec3 viewingDirection = normalize(fragPos);
    float lightness = dot(viewingDirection, reflectedDirection);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```

oog is in de oorsprong

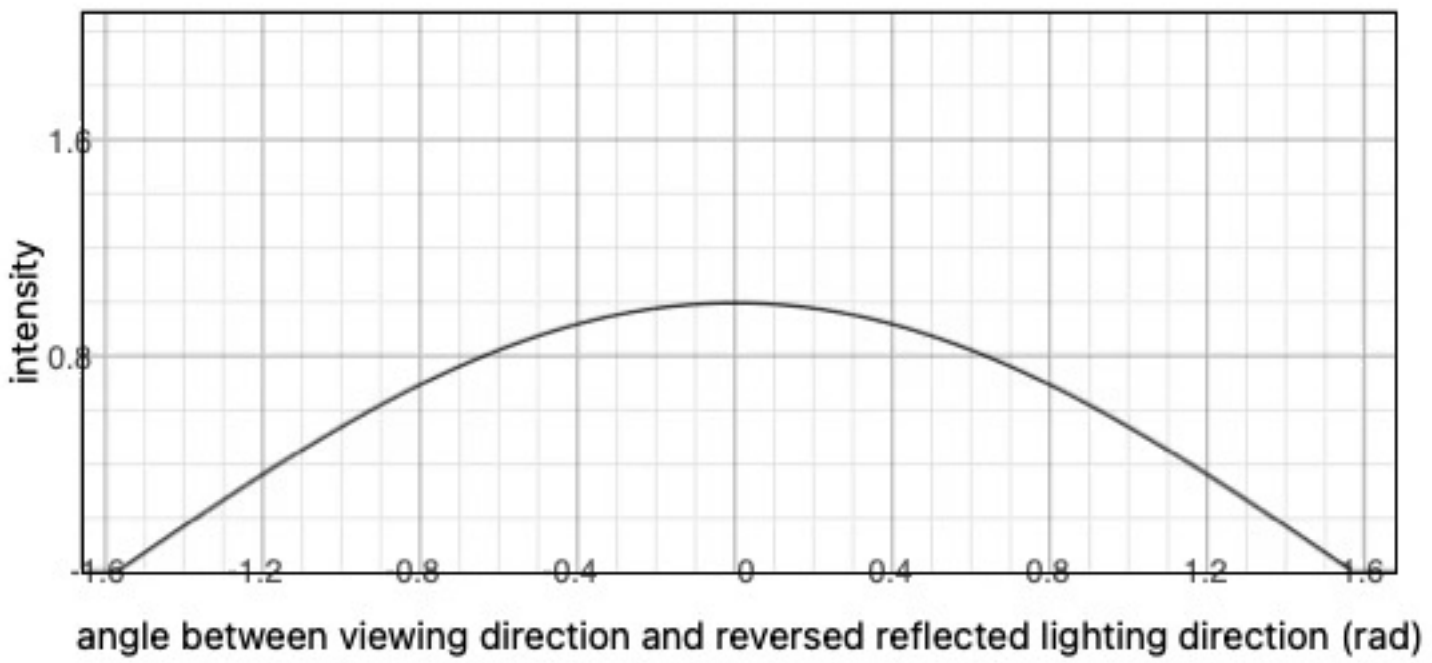


Specular lighting

- Glimvlek is erg groot; niet heel realistisch.
- Meer concentreren: gebruik machtsfunctie:

$$f(x) = x^a$$

- a: *shinyness* (1…32):



Specular lighting: shinyness

- In GLSL: `pow(x, a)`.
- Let op: alleen positieve getallen; gebruik `max(0, x)`.

```
⋮
void main() {
    vec3 normalizedNormal = normalize(normal);
    vec3 reverseLightDirection = normalize(lightPos - fragPos);
    vec3 reflectedDirection = reflect(reverseLightDirection,
      normalizedNormal);
    vec3 viewingDirection = normalize(fragPos);
    float lightness = pow(max(0.0f, dot(viewingDirection,
      reflectedDirection)), 32.0f);
    fragColor = baseColor;
    fragColor.rgb *= lightness;
}
```



Opdracht 2: belichting

- **Zie instructies op GitLab**.

- Combineer de 3 types belichting met een kubus met kleur en texture.



Check-out

- Wat hebben we geleerd?
  - Hoe je textuur toevoegt aan een vorm in een shader.
  - Welke soorten belichting er zijn en hoe die wiskundig werken.
  - Hoe je belichting toevoegt aan een shader.



Afsluiting

Verwerkingsopdracht:

- Leidt de reflectieformule $\vec{r} = \vec{l} - 2(\vec{n} \cdot \vec{l})\vec{n}$ af. Gegeven de normaalvector $\vec{n}$ en de omgekeerde lichtrichting $\vec{l}$, schrijf stap voor stap op hoe je de gereflecteerde vector $\vec{r}$ bepaalt. **Hint**: projecteer $\vec{l}$ op $\vec{n}$ (zie les 2). Dit is afstand van het oppervlak naar de lichtbron, haaks op het oppervlak. Als je nu tweemaal de omgekeerde verplaatsing neemt vanaf de lichtbron, kom je uit op het gevraagde gereflecteerde punt (maak een tekening).

Voorbereidingsopdracht:

- Bereid een korte uitleg voor (max. 5') over elk van de volgende onderwerpen:
  - Texture mapping in OpenGL en de betekenis van *u*, *v*-coördinaten
  - Diffuse belichting (directional en positional)
  - Speculaire belichting, inclusief shinyness

Volgende keer:

- Materiaaleigenschappen

HOGESCHOOL ROTTERDAM

Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden